07-1-secrets-delivery

accounts-api security assessment · generated from the repository, not hand-edited

```
# 07-1-secrets-delivery.yaml
#
# Section 7 deliverable: the secrets delivery configuration. No plaintext
# credential in the repo or in any committed manifest; mapped to AWS Secrets
# Manager via the Secrets Store CSI driver, with External Secrets Operator
# shown as the alternative and the trade-off stated. The IAM half is 07-4.
#
# Assembled by scripts/build-submission.py from the repository.

# ============================================================================
# SOURCE: k8s/externalsecret.yaml
# ============================================================================
# Section 6 — delivery path for the database credential. Threat T3.
#
# Path, end to end:
#
#   AWS Secrets Manager (prod/accounts-api/db, KMS CMK alias/accounts-api-secrets)
#     └─ read by the pod's IRSA role, via the Secrets Manager VPC endpoint
#          └─ Secrets Store CSI driver mounts it as a file in the pod
#               └─ app reads /var/run/secrets/db/credential.json and re-reads on change
#
# Nothing in this repository, and nothing in any committed manifest, contains the
# credential or a reference that would yield it without the pod's IAM identity.
# What is committed is the *name* of the secret, which is not a secret.
#
# Two mechanisms are shown. The CSI driver is what ships; External Secrets
# Operator is included because it is the more common answer and the trade-off is
# worth being explicit about — see the note before the ExternalSecret below.
---
apiVersion: v1
kind: Namespace
metadata:
  name: accounts
  labels:
    kubernetes.io/metadata.name: accounts
    pod-security.kubernetes.io/enforce: restricted      # PSA as the floor under Kyverno
    pod-security.kubernetes.io/enforce-version: latest
---
# WHAT SHIPS. The CSI driver mounts the secret straight into the pod's tmpfs.
# The credential never becomes a Kubernetes Secret, so:
#   * it is never in etcd, so a compromised etcd backup does not contain it
#   * `get secrets` in ns/accounts does not yield it — which is why the
#     remediated RBAC in section 4a could drop secret access entirely
#   * rotation propagates to a running pod without a restart (rotationPollInterval)
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: accounts-api-db
  namespace: accounts
spec:
  provider: aws
  parameters:
    region: ap-south-1
    objects: |
      - objectName: "arn:aws:secretsmanager:ap-south-1:123456789012:secret:prod/accounts-api/db"
        objectType: "secretsmanager"
        objectAlias: "credential.json"
```

```
        # AWSCURRENT explicitly. During rotation both AWSCURRENT and AWSPREVIOUS
        # exist; pinning the stage is what makes "rotation completed" mean the
        # old credential is genuinely out of use.
        objectVersionLabel: "AWSCURRENT"
  # Deliberately NOT set: `secretObjects`. That field syncs the mounted file back
  # into a Kubernetes Secret for convenience, and doing so would undo every
  # advantage listed above. It is the single most common misconfiguration of this
  # driver.
---
# THE ALTERNATIVE, for comparison. ESO is easier to adopt and works with any
# workload without a CSI mount, but it *materialises a Kubernetes Secret* — so the
# credential lands in etcd and anyone with `get secrets` in the namespace has it.
# On a cluster shared with a dozen teams that is a meaningfully worse position,
# which is why the CSI driver wins here. Kept in the repo because for the KYC API
# key — lower value, needed by a workload that cannot take a CSI mount — ESO is
# the right trade.
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secretsmanager
  namespace: accounts
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-accounts   # its own SA, its own IRSA role
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: accounts-api-kyc
  namespace: accounts
spec:
  refreshInterval: 1m        # bounds propagation delay after an emergency rotation
  secretStoreRef:
    name: aws-secretsmanager
    kind: SecretStore
  target:
    name: accounts-api-kyc
    creationPolicy: Owner
    deletionPolicy: Delete   # revoking the ExternalSecret revokes the Secret
  data:
    - secretKey: kyc_api_key
      remoteRef:
        key: prod/accounts-api/kyc
        property: api_key
```